

# Resolving deadlocks in video game logic using CPN Tools

Dag Åsmund Ørnes

Western Norway University of Applied Sciences, Inndalsveien 28, 5063 Bergen, Norge  
599061@stud.hvl.no

**Abstract.** Colored Petri Nets are used to analyze concurrent and distributed systems. This project used CPN Tools to simulate and analyze a concurrent system in video games, namely a branching quest system in a single-player role-playing game, with the goal to investigate whether CPNs can be used to identify and prevent deadlocks in video game logic. The quest system will be modeled in CPN Tools, analyzed for potential deadlocks, and then implemented in Unity game engine. This project aims to demonstrate that model-based engineering techniques, well established in traditional software development, can also provide practical value in video game development.

**Keywords:** Petri Nets · Colored Petri Nets · CPN Tools · State Space · Deadlock · Unity.

## 1 Introduction

### 1.1 What are Colored Petri Nets?

Petri nets is a formalized modeling language consisting of places and transitions. It is a bipartite graph system of places, transitions and edges (also called arcs). Transitions represent the process between places, and can be used to model dependencies between places. Petri Nets are useful for the analysis of concurrent, discrete-event dynamic systems, and to identify potential deadlocks within that system, done by observing the flow of tokens in the system. [1]

Colored Petri Nets (CPNs) combine the capabilities of Petri Nets and that of a high-level programming language. [2] Tokens in a CPN are assigned a value (color) that can change as it flows through the system. This gives us the ability to model systems with more complex conditional logic, like gateways that only let through tokens with a desired value or other conditional logic.

### 1.2 CPNs and video game logic

Video Games contain a large amount of concurrent systems. What might come to mind first, is Multiplayer Games, where the game programmers have to account for a large amount of inputs from several different players at the same time.

Singleplayer Games also contain concurrent systems. Video Games like Role-Playing Games (RPGs), often contain complex Branching Quest Systems, where you as the player, can complete a task often in several different ways, with several different outcomes.

Imagine a scenario where you as the player character are sent out to collect an item for a kind, old woman, but then a shady character offers you a nice sum of money to give that same item to him, instead of the kind, old woman. This is now a branching quest with different outcomes; we can choose to give the item to the shady character or the kind, old woman. We might even lose the item somehow, or the item might be fragile, and able to be destroyed. Maybe the item is medicinal and the kind, old woman need it urgently, or else she will die, introducing a time limit to the quest. Quests like these are everywhere in Video Games today, and if not handled correctly, can lead to game breaking bugs, where the quest end up in a deadlock, meaning unable to reach either success- or fail-state, making the player unable to continue the game. This project aims to investigate if CPNs can be used to analyze these kinds of systems and identify, and thereby resolve potential deadlocks.

### 1.3 CPN Tools

CPN Tools is a software for editing, simulating, and analyzing Colored Petri Nets. CPN Tools was originally developed by the CPN Group at Aarhus University in Denmark from 2000 to 2010, when it was transferred to the AIS group, Eindhoven University of Technology in the Netherlands. In 2015, CPN Tools was discontinued and replaced with CPN IDE, but for this project, CPN Tools will be used.

## 2 Motivation

Trasorbares et al. writes that Model-Based Engineering is a large field in traditional software development, but is not as much explored in the field of video game development [3]. This project therefore aims to bring a concept, well known in traditional software systems and distributed systems, namely CPNs, into the field of video game development. By using CPN Tools to simulate and analyze a concurrent system well known in the world of video games, a multi-branching quest system, this project aims to investigate if the use of CPN Tools is a feasible way of preventing bugs in video games, and help make video games more reliable.

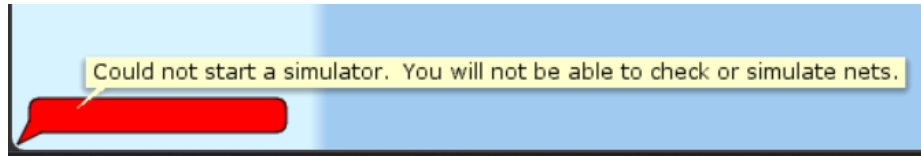
## 3 Method

In this project a multi-branch quest will be modeled and simulated using CPN Tools. The quest will then be analyzed to ensure that there are no possible deadlocks in the system. The same quest will then be implemented in the game engine Unity. While Unity will bring a visually rich representation of the quest system, the focus of this project will be on the use of CPN Tools.

## 4 Design and Implementation

### 4.1 Using CPN Tools

Having been discontinued since 2015, running CPN Tools posed some challenges. One is that it is missing Microsoft certificates used by Windows Security to judge software and apps trustworthy. The "Smart App Control" function inside Windows Security had to be disabled in order to run CPN Tools. When CPN Tools finally started up, there was a bug tied to CPN Tools and Cygwin. This bug made it so the syntax checking did not work correctly, accompanied by a vague error message akin to "something is wrong" seen in figure 1, making it so the simulation did not work. Luckily this was a known issue on the CPN Tools web site, so a number of possible solutions were found on the FAQ page. These ranged from restarting the computer, and "run as administrator" to deleting keys from the windows registry. One solution that seems to have worked in our case, was to re-install CPN Tools in a directory with a name without spaces, meaning changing the install directory from the default "CPN Tools" to "CPNTools". The

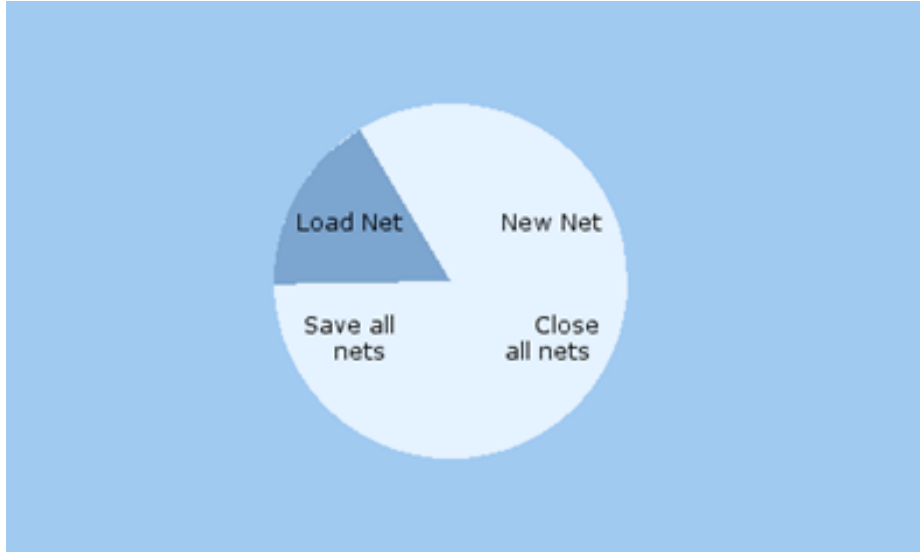


**Fig. 1.** This shows the vague error message telling the user essentially that "something" is wrong.

GUI is not like what we are used to today. Where there is a menu of editing tools on the side, choosing which to use is not done by simply clicking on them, as one would think. To choose a tool, one first have to click and hold on a tool category in the side menu, drag it to where you want it and release. A small window the opens up containing a collection of tools in that category, like "create place", "create transition", "create arc", and so on. The act of saving and loading nets along with other contextual functions is done by clicking and holding down right mouse button on the empty space on the left side of the screen. This opens up a "radial menu" with different choices as seen in figure 2, and to select an option, one drags the cursor to that option and releases the right mouse button.

### 4.2 Unity Project Setup

A basic 3D Unity project was setup early in the process to support the branching quest logic modeled in CPN Tools. The Unity project contained a simple scene with a player character, an "Old Lady" character, a "Shady" character, and an item to be picked up and delivered as part of the quest (see figure 3). A third-person camera follows the player character, and the user moves the player



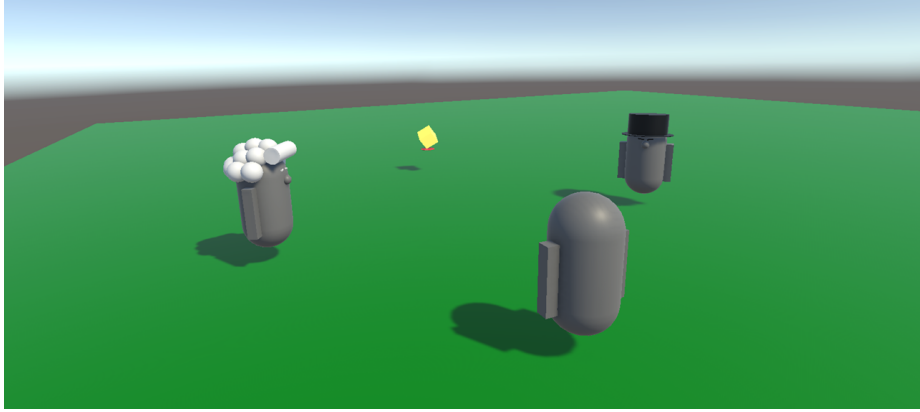
**Fig. 2.** The radial menu showing a few options to choose from.

character with the W, A, S, D keys on the keyboard and can look around by moving the camera with the mouse. The ground was made with a simple green plane resembling grass. Every asset in the scene was constructed using Unity's own primitive 3D shapes, like capsules (oblong sphere shape), boxes, and spheres. To distinguish between the characters in the scene (see figure 4), the Old lady character was given something resembling white, curly hair, the shady character was given a black top hat and black squinting eye brows, while the player character was left plain, as we only see the back of it anyway.

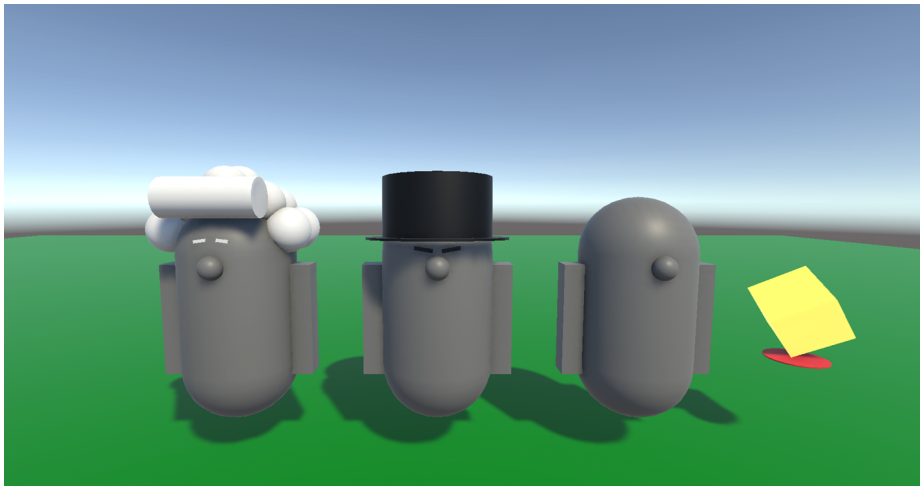
### 4.3 CPN model - version 1

The first version of the quest modeled in CPN Tools (figure 5), has one token going through a series of transitions, first setting the quest to active and then eventually finding the item. There is a chance for the item to be lost, leading the token through the "LoseItem" transition and on to the QuestFailed place. If the item is not lost, the token will go on to DeliverChoice place via either the ChooseOldWoman- or ChooseShadyCharacter-place, and onto the CompleteDelivery transition to QuestComplete.

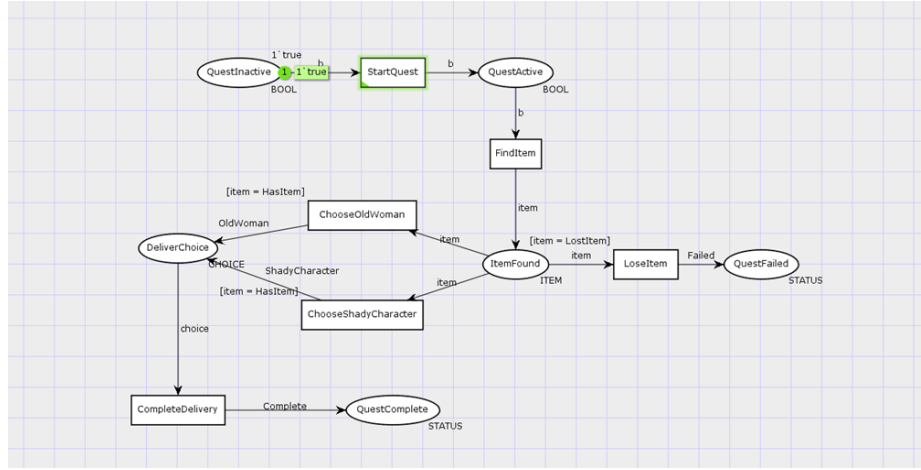
Due to lack of experience with CPN Tools, we were afraid to create an overly complex design, and end up with nothing to show for it at the end. Because of this, version 1 might have ended up on the other end of the scale as an overly simple design, only containing one token, 6 transitions, 6 places, and 12 arcs. The design also contained only two end states (QuestComplete and QuestFailed), meaning the perceived choice of giving the item to the old lady, or the shady character leads to the same end.



**Fig. 3.** Overview of the Unity scene showing the starting positions of the characters and the item.



**Fig. 4.** All characters lined up in the Unity Scene. from left to right: Old Lady, Shady Character, Player Character and the Item.



**Fig. 5.** First version of quest CPN.

#### 4.4 CPN Model - version 2

The lack of complexity present in version 1 of the CPN model, was addressed in version 2 as seen in figure 6. In version 1, there is a linear path from QuestInactive to QuestActive, whereas in version 2 the player has the option to either refuse or accept the quest. The model starts out in the QuestInactive place, where a boolean token representing the player flows through the ApproachLady transition to the TalkingToLady place. From TalkingToLady, the token can go to one of two transitions, either AcceptQuest or RefuseQuest. Refusing the quest sends the token back to start, where it can approach the lady again, but going via the RefuseQuest transition, will set a Refusals token in motion. The refusals token will iterate by one every time the RefuseQuest transition fires. After two iterations, The RefuseQuest transitions locks, and another transition called RefuseQuestFinal will unlock, meaning with a path leading to the QuestRefused fail state. In plain speak this means that while the player has the option to either accept or refuse the quest, upon the third refusal, the quest will fail even before it starts.

From the TalkingToLady place, the player token can go through the AcceptQuest transition onto the QuestActive place, then through the FindItem transition to the ItemFound place. The player now has the quest item in their possession, and can either keep hold of the item and go on to deliver it, or they can lose the item. Losing the item sends the player token through the LoseItem transition and to the QuestFailed fail state.

If the item is not lost, the player has the option to deliver the item to either the old lady or the shady character. The token can go through either the ChooseOldLady transition or the ChooseShadyCharacter transition and to the DeliverChoice place. If the token went through the ChooseOldLady transition, the token will continue through the DeliverToOldLady transition and on to the

QuestCompleteGood success state. If the token reached the DeliverChoice place, it will continue on through the DeliverToShadyCharacter transition and on to the QuestCompleteBetrayal success state. By going through the DeliverToShadyCharacter transition another token will fire through that same transition from the LadyStatus place and on to the LadyDied place. In plain speak this means that if the player delivers the item to the old lady, the quest gets a “good ending”, but if the player delivers the item to the shady character, the quest gets a “bad ending” and the old lady dies. Version 2 almost doubled in size from version 1, containing 12 places, 10 transitions and 23 arcs. The new design now contains four end states (reachable by the player token), plus one end state for LadyStatus, that is reached if DeliverToShadyCharacter fires, sending a token from LadyStatus to LadyDied.

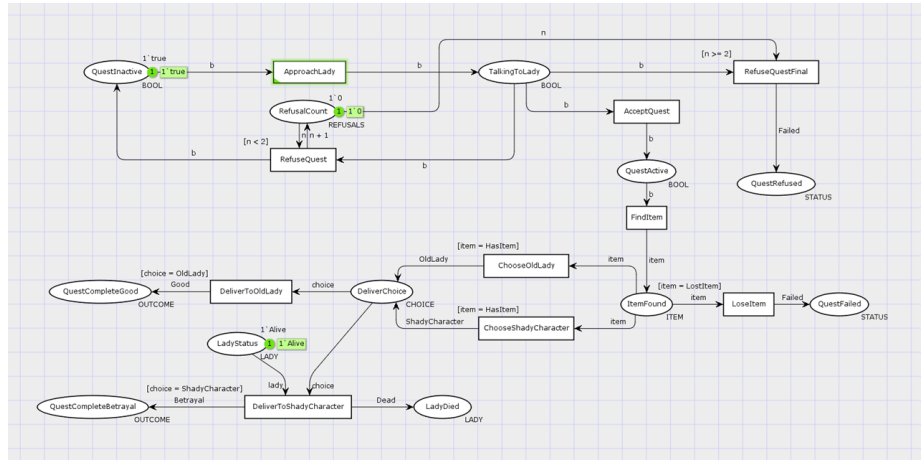


Fig. 6. Second version of the quest CPN with added complexities.

#### 4.5 From CPN Tools to Unity

The thesis of this project is that modeling game logic in CPN Tools, and then programming the same logic in Unity, can lead to more reliable video games with fewer bugs. One challenge is how we translate the model from CPN Tools to Unity while retaining the structural integrity of the CPN model, meaning that if we have to program the same logic from scratch when moving to Unity, we can cast doubts on the necessity of CPN Tools in this process, and the validity of the thesis of this project.

**Exporting Java Code** CPN Tools since version 4.00 have the function to export model as Java code, as seen in figure 7. At the start of this project we

therefore had a plan to export the model as Java code, manually convert it to C Sharp, as these languages are fairly similar, and then implement the code in the Unity project.

In practice, this did not work as hoped. First, the Java function did not work. Not surprising, since CPN Tools have not been supported since 2015. Second, after conducting more research into CPN Tools, the Java function does not export complete executable Java code, as first thought, but rather minimal stubs with boilerplate main methods, making this feature useless for our intent.

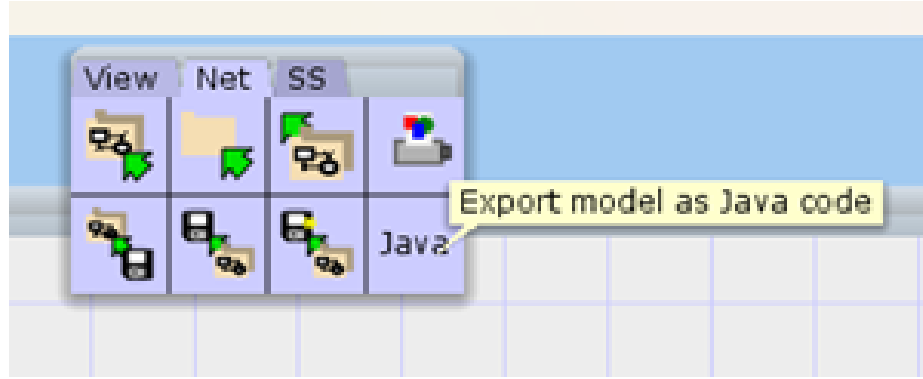


Fig. 7. Tool in CPN Tools to export models as Java code.

**Exporting XML** When saving nets in CPN Tools, they are written to .cpn files, which are XML based. After a lot of back and forth with Claude AI, We managed to create a C Sharp script in Unity that is able to parse the .cpn file into classes for arcs, places, transitions and so forth, making CPN Tools objects into relevant concepts able to be manipulated in Unity.

## 5 Evaluation

### 5.1 Evaluating the CPN

By using the tool "Calculate State Space" on a CPN inside CPN Tools and then "Save Report", we get a detailed analysis of the CPN. This report contains information about the total amount of states the system can be in and the transitions between them. It also shows the number of end states, cycles, infinite loops, dead nodes, and potential deadlocks.

Table 1 shows the statistics portion of the State Space report from both versions of the CPNs. Comparing Version 1 (V1) and Version 2 (V2) in the State Space column, we see that V1 contains 8 nodes and 8 arcs, while V2 contains 31 nodes and 30 arcs. Nodes show the total amount of states the quest can be in,



and arcs represent the transitions between these states. The design near doubling in size from V1 to V2 (mentioned in section 4.4), leads to a significant increase in complexity, with more than triple the amount of states from 8 in V1 to 31 in V2. "Secs" and "Full" are the same for both versions. "Secs" is the time spent to calculate the State Space in seconds, meaning it was almost instantaneous, and our increase in complexity has not impacted the calculation time. "Full" tells us the analysis is complete, and we have the whole picture. When looking at both versions in the SCC Graph column, we see that the values for each version is the same as in the State Space column, meaning every state is it's own strongly connected component, meaning no state can loop back to a previous one, and as such the quest always moves forward, never backwards.

**Table 1.** State Space report on Version 1 and Version 2 both analyzed as a State Space and a SCC Graph.

|        | State Space |           | SCC Graph |           |
|--------|-------------|-----------|-----------|-----------|
|        | Version 1   | Version 2 | Version 1 | Version 2 |
| Nodes  | 8           | 31        | 8         | 31        |
| Arcs   | 8           | 30        | 8         | 30        |
| Secs   | 0           | 0         | 0         | 0         |
| Status | Full        | Full      | n/a       | n/a       |

The Boundedness portion of the State Space Report tells us something about the flow of tokens through the system. Looking at the full State Space reports in appendix A and B, we can see that all upper bound are 1 and lower bound are 0, meaning no place ever hold more than 1 token, showing that we do not have any accumulation of tokens in our system. If we look further down in the V2 report, "Best Upper Multi-set Bounds" go into more detail, showing for example that RefusalCount can hold 0,1, or 2, as intended by the  $[n < 2]$  guard, and DeliverChoice can hold either OldLady or ShadyCharacter, meaning both branches are reachable. "Best Lower Multi-set Bounds" show "empty" in all places, meaning every place can be empty, and no place is permanently occupied, showing that tokens flow through the system rather than getting stuck.

The last sections, Home Properties, Liveness Properties and Fairness Properties, describe the systems as a whole. Rather than describing the behavior of our system, these sections tell us about the outcome of these behaviors, like infinite cycles, dead transitions, and deadlocks. Both versions show mostly the same results, except for the line "Dead Markings". Looking at figure 8, from the version 2 report, we see that Home Markings show "None". This means there is no state the system can always return to, so once a terminal state is reached, it stays there forever. This is intentional, making the quest non-repeatable.

Dead Markings differ from V1 to V2. In V1 Dead marking have a value of "2 [5, 8]", meaning it has 2 terminal states, one with a token in QuestFailed and one with a token in QuestComplete, respectively. Looking at V2, we see that Dead Markings have a value of "10 [31, 30, 27, 26, 25,...]". This is because

in V2 we designed the quest with 4 different endings. Accepting the quest lead down a path to 3 potential endings, and consequently 3 terminal states. These 3 states can be reached by accepting the quest outright, but it they are also reachable after refusing the quest once, adding 3 more terminal states, or twice, again adding 3 more terminal states, adding up  $3 + 3 + 3 = 9$ . The last terminal state is reached by refusing the quest a third time,  $9 + 1 = 10$ .

Dead Transition Instances has the result "None", meaning every transition fires at least once, and therefore no place is unreachable and therefore redundant.

Live Transition Instances also have the result "None". This means that no transitions in the system fires indefinitely, and consequently there are no infinite loops.

In the "Fairness Properties" section, it says "No infinite occurrence sequences". This explicitly confirms that there are no infinite loops, and the quest always terminates. This is important for a video game system to ensure that it can never become permanently stuck in a cycle.

```

Home Properties
-----

Home Markings
  None

Liveness Properties
-----

Dead Markings
  10 [31,30,27,26,25,...]

Dead Transition Instances
  None

Live Transition Instances
  None

Fairness Properties
-----

  No infinite occurrence sequences.
```

**Fig. 8.** Screenshot of the lower sections in the State Space Report from version 2

## 5.2 Evaluating Unity implementation

Programming logic in Unity and CPN Tools are two very different things. Programming Unity while thinking of tokens flowing through the system is highly unusual. One way translate CPNs to Unity and generally object-oriented programming languages like C Sharp, is to think of transitions like methods that

can only be called once, and also with preconditions, meaning in order for methods to be called, certain other methods need to have been called before. This means that every method needs to store the state of the system in a way to communicate which methods are able to be called next. This makes sense if you think about figure 6, where the token starting in QuestInactive, cannot go directly to FindItem, but rather it has to flow through first ApproachLady, then AcceptQuest, before it can go on to FindItem and beyond.

When implementing the CPN in Unity, we solicited a lot of help from Claude AI. It did a great job in mimicking the token and transition logic from CPN into C Sharp, translating transitions from CPN to methods in C Sharp as seen in listing 1.1, making it simple to call the methods in our game logic, while keeping track of the different states, and also preventing us from calling a method corresponding to an unreachable transition.

**Listing 1.1.** Code snippet from QuestManager.cs with methods mimicking CPN logic.

```
/// Player walks up and talks to the old lady -> fires ApproachLady
public void ApproachLady() => FireByName("ApproachLady");

/// Player accepts the quest -> fires AcceptQuest
public void AcceptQuest() => FireByName("AcceptQuest");

/// Player refuses the quest -> fires RefuseQuest
public void RefuseQuest() => FireByName("RefuseQuest");

/// Player refuses the quest for the last time -> fires RefuseQuestFinal
public void RefuseQuestFinal() => FireByName("RefuseQuestFinal");
```

These methods are then called in the PlayerInteraction script that governs the player's interaction with the game world. in listing 1.2, this happens on the lines where the questManager object is called.

**Listing 1.2.** Code snippet from PlayerInteraction.cs where transitions-mimicking methods are called.

```
if (!questManager.IsQuestInProgress())
{
    if (currentRefuseCount >= refuseLimit)
    {
        finalRefuseDialogue.text = "OldLady:\nPLease, little one. I really really need it";
    }
    questManager.ApproachLady();
    questManager.IsTalkingToLady();
    questStatus.text = "Accept or Refuse the quest from the Old Lady";
    questAcceptance.SetActive(true);
    Cursor.visible = true;
    Cursor.lockState = CursorLockMode.None;
}
else if (questManager.HasItem())
{
    deliverOldLadyPanel.SetActive(true);
    Cursor.visible = true;
    Cursor.lockState = CursorLockMode.None;
}
```

However, when implementing the logic for failing the quest if the player refuse it a certain amount of times, the Claude AI solution got overly complex as seen in listing 1.3. It sticks very close to the CPN logic of a separate token gets fired every time the RefuseQuest transition fires incrementing "n" by 1 every time until it satisfies the "[n >= 2]" guard of RefuseQuestFinal, sending the player token down this path and to the QuestRefused ending on the next Refuse.

**Listing 1.3.** Complex RefuseCounter solution given by Claude AI.

```
public void RefuseQuest() {
```

```

var refusalPlace = net.GetPlaceByName("RefusalCount");
var talkingPlace = net.GetPlaceByName("TalkingToLady");

if (refusalPlace == null || talkingPlace == null) return;
if (talkingPlace.tokens < 1) {
    Debug.LogWarning("RefuseQuest: not talking to lady");
    return;
}

int n = refusalPlace.tokens; // tokens = current refusal count

if (n < 2) {
    // RefuseQuest transition: n < 2, increment count, return to TalkingToLady
    refusalPlace.tokens = n + 1;
    // TalkingToLady token stays (player can try again)
    Debug.Log($"Refused quest. Refusal count: {refusalPlace.tokens}");
} else {
    // RefuseQuestFinal transition: n >= 2, quest fails
    talkingPlace.tokens--;
    refusalPlace.tokens = n + 1;
    var failPlace = net.GetPlaceByName("QuestFailed");
    if (failPlace != null) failPlace.tokens++;
    Debug.Log("RefuseQuestFinal fired, Quest Failed!");
}
}

public int GetRefusalCount() =>
    net.GetPlaceByName("RefusalCount")?.tokens ?? 0;

public bool IsRefuseFailed() =>
    IsQuestFailed() && !IsQuestInProgress();

```

A simpler solution developed by us, can be seen in listing 1.4, where the logic is implemented using a simple if-else statement. If the variable `currentRefuseCount` is lower than `refuseLimit`, then `currentRefuseCount` is incremented and `RefuseQuest()` is called. Otherwise, `RefuseQuestFinal()` is called, failing the quest. Using this method we deviate slightly from the CPN logic, while still retaining the integrity of the system by calling transition methods in `QuestManager.cs`, namely `RefuseQuest()` and `RefuseQuestFinal()`.

**Listing 1.4.** Simple RefuseCounter solution developed by authors.

```

public void RefuseQuest()
{
    if (currentRefuseCount < refuseLimit)
    {
        currentRefuseCount++;
        questManager.RefuseQuest();
        refuseCount.text = "Refusals: "
        + currentRefuseCount
        + "/"
        + refuseLimit;
    }
    else
    {
        questManager.RefuseQuestFinal();
        EndState("Refused");
        refuseCount.color = Color.red;
    }
}

```

## 6 Discussion

While using the XML parser in unity to consult the CPN for every action in the quest logic prevents and allows certain actions according to the CPN, it still requires a lot of tweaking in Unity and the C sharp scripts to get the logic to work. We experienced situations where we had to abandon the CPN logic in favor of simpler C Sharp logic, as explained in section 5.2. Too much of these kind of workarounds might compromise the integrity gathered from CPN Tools and it's State Space Report, making the Unity implementation susceptible to

the kinds of bugs CPN Tools are meant to prevent. In our example, however, as seen in listing 1.4, we still keep the implementation tethered to the CPN logic by calling the methods in "questManager". It is difficult to determine the threshold at which the workarounds would compromise the integrity without further study. The workarounds were minor in our case, but it is reasonable to assume that as complexity grows, the amount of necessary workarounds would also grow. At some point the Unity implementation might diverge enough from the CPN model that the results from the State Space Report no longer reliably apply to the actual game. Determining this threshold would require testing with significantly more complex models. This is a good idea for future work.

We could argue that we do not necessarily need to translate the CPN to Unity in the matter we have, to ensure the integrity of the system. This is not an argument for abandoning CPN altogether in favor of something simple akin to UML, however. One of the most useful functions of CPN Tools is the ability to generate a State Space Report. This function tells us explicitly if we have faults in our system like deadlocks, redundancies or infinite loops. It is then possible to implement the solution independently in Unity, while following the CPN as a blueprint, without connecting it directly with a parser. It would at least be possible in our case, but it is unclear how easy it would be when the complexity of the CPN model grows.

## 7 The use of AI in this project

Claude AI has been used frequently during this project. It has been used as a tool to help in the CPN designs, translation from CPN to Unity, and general Unity assistance. Claude AI was invaluable at the start of the project, not only because of our unfamiliarity with the concept of CPNs, but with CPN Tools. CPN Tools' unorthodox UI would have taken much longer to get familiar with using only the documentation available on their website and youtube. By using Claude AI, we quickly got an understanding of the different tools within CPN Tools and could start designing and tweaking CPNs fairly quickly. We found it most rewarding consulting Claude AI by giving it screenshots of the model, as it frequently misunderstood what was being shown, and while giving some good feedback, it also gave wrong ones, meaning we had to come up with much more of the solution ourselves, rather than getting the complete solution presented to us. On the other hand, if we got completely stuck, we could feed it the .cpn file containing the XML, and this way it was more reliable to give us the complete, working solution. As an example, we used the latter approach once when making the State Space Reports. The CPN worked without errors in CPN Tools, but when we generated the State Space Report, it said we had only 1 node, and 0 arcs, meaning it only detected one state, and could not explore further. Claude AI analyzed the .cpn file and found that there was a white space on one of the tokens in CPN Tools. After removing the white space, the State Space Report worked correctly. It is impossible to say how long it would have taken to detect this error on our own.

Using Claude AI to create an XML Parser and some logic translate the CPN logic to Unity, was also invaluable as it saved us valuable time and allowed us to implement a complete Unity solution with all the logic present in the CPN.

Claude AI was also consulted on the report writing process at certain points of the project, helping with grammar, structure and keeping the content relevant. However, apart from the code snippets in this report, nothing has been directly copied from AI. Everything has been written by the authors.

## 8 Conclusion

We modeled two versions of a multi-branch quest system in CPN Tools. We then checked the integrity by generating a State Space Report. This report confirmed no deadlocks before implementation. With the help of Claude AI, we then implemented the second version in a Unity game using an XML parser to translate components directly from the .cpn file into C Sharp code. The CPN model served as a reliable blueprint during Unity development, though direct integration required workarounds. The end result was a short, but fully functioning video game where the player can partake in a quest with multiple outcomes, none of them being a deadlock.

## References

1. Freeman, T.: Petri nets, basis of the flow of tokens (2015), <https://www.youtube.com/watch?v=1IPOIE0PvQY>
2. Jensen, K., Kristensen, L.M.: Coloured Petri Nets: Modelling and Validation of Concurrent Systems. Springer Berlin Heidelberg (2009). <https://doi.org/https://doi.org/10.1007/b95112>
3. Trasobares, J.I., África Domingo, Echeverria, J., Arcega, L., Cetina, C.: Deepening our understanding on the use of models and code in game software engineering: A controlled experiment in unreal engine. 2025 ACM/IEEE 28th International Conference on Model Driven Engineering Languages and Systems (MODELS) pp. 107–118 (2025). <https://doi.org/10.1109/MODELS67397.2025.00016>

## A State Space Report v1

CPN Tools state space report for:  
/cygdrive/D/CPNTools/SavedNets/DAT355projectnet.cpn  
Report generated: Tue May 5 18:58:35 2026

Statistics

---

State Space

Nodes: 8  
Arcs: 8  
Secs: 0  
Status: Full

Scc Graph

Nodes: 8  
Arcs: 8  
Secs: 0

Boundedness Properties

---

Best Integer Bounds

|                            | Upper | Lower |
|----------------------------|-------|-------|
| OldLadyQuest'DeliverChoice | 1     | 0     |
| OldLadyQuest'ItemFound     | 1     | 0     |
| OldLadyQuest'QuestActive   | 1     | 0     |
| OldLadyQuest'QuestComplete | 1     | 0     |
| OldLadyQuest'QuestFailed   | 1     | 0     |
| OldLadyQuest'QuestInactive | 1     | 0     |

Best Upper Multi-set Bounds

|                            |   |
|----------------------------|---|
| OldLadyQuest'DeliverChoice | 1 |
| 1`OldWoman++               |   |
| 1`ShadyCharacter           |   |
| OldLadyQuest'ItemFound     | 1 |
| 1`HasItem++                |   |
| 1`LostItem                 |   |
| OldLadyQuest'QuestActive   | 1 |
| 1`true                     |   |
| OldLadyQuest'QuestComplete | 1 |

1`Complete  
OldLadyQuest'QuestFailed 1  
1`Failed  
OldLadyQuest'QuestInactive 1  
1`true

Best Lower Multi-set Bounds  
OldLadyQuest'DeliverChoice 1  
empty  
OldLadyQuest'ItemFound 1  
empty  
OldLadyQuest'QuestActive 1  
empty  
OldLadyQuest'QuestComplete 1  
empty  
OldLadyQuest'QuestFailed 1  
empty  
OldLadyQuest'QuestInactive 1  
empty

#### Home Properties

---

Home Markings  
None

#### Liveness Properties

---

Dead Markings  
[5,8]

Dead Transition Instances  
None

Live Transition Instances  
None

#### Fairness Properties

---

No infinite occurrence sequences.



## **B State Space Report v2**

CPN Tools state space report for:  
 /cygdrive/D/CPNTools/SavedNets/DAT355projectnet\_v2.cpn  
 Report generated: Wed May 6 18:46:40 2026

## Statistics

---

### State Space

Nodes: 31  
 Arcs: 30  
 Secs: 0  
 Status: Full

### Scc Graph

Nodes: 31  
 Arcs: 30  
 Secs: 0

## Boundedness Properties

---

### Best Integer Bounds

|                                      | Upper | Lower |
|--------------------------------------|-------|-------|
| OldLadyQuest'DeliverChoice 1         | 1     | 0     |
| OldLadyQuest'ItemFound 1             | 1     | 0     |
| OldLadyQuest'LadyDied 1 1            | 1     | 0     |
| OldLadyQuest'LadyStatus 1            | 1     | 0     |
| OldLadyQuest'QuestActive 1           | 1     | 0     |
| OldLadyQuest'QuestCompleteBetrayal 1 | 1     | 0     |
| OldLadyQuest'QuestCompleteGood 1     | 1     | 0     |
| OldLadyQuest'QuestFailed 1           | 1     | 0     |
| OldLadyQuest'QuestInactive 1         | 1     | 0     |
| OldLadyQuest'QuestRefused 1          | 1     | 0     |
| OldLadyQuest'RefusalCount 1          | 1     | 0     |
| OldLadyQuest'TalkingToLady 1         | 1     | 0     |

Best Upper Multi-set Bounds

```
OldLadyQuest'DeliverChoice 1
    1`OldLady++
1`ShadyCharacter
    OldLadyQuest'ItemFound 1
        1`HasItem++
1`LostItem
    OldLadyQuest'LadyDied 1
        1`Dead
    OldLadyQuest'LadyStatus 1
        1`Alive
    OldLadyQuest'QuestActive 1
        1`true
    OldLadyQuest'QuestCompleteBetrayal 1
        1`Betrayal
    OldLadyQuest'QuestCompleteGood 1
        1`Good
    OldLadyQuest'QuestFailed 1
        1`Failed
    OldLadyQuest'QuestInactive 1
        1`true
    OldLadyQuest'QuestRefused 1
        1`Failed
    OldLadyQuest'RefusalCount 1
        1`0++
1`1++
1`2
    OldLadyQuest'TalkingToLady 1
        1`true
```

Best Lower Multi-set Bounds

```
OldLadyQuest'DeliverChoice 1
    empty
OldLadyQuest'ItemFound 1
    empty
OldLadyQuest'LadyDied 1
    empty
OldLadyQuest'LadyStatus 1
    empty
OldLadyQuest'QuestActive 1
    empty
OldLadyQuest'QuestCompleteBetrayal 1
    empty
OldLadyQuest'QuestCompleteGood 1
    empty
OldLadyQuest'QuestFailed 1
    empty
```

OldLadyQuest'QuestInactive 1  
empty  
OldLadyQuest'QuestRefused 1  
empty  
OldLadyQuest'RefusalCount 1  
empty  
OldLadyQuest'TalkingToLady 1  
empty

#### Home Properties

---

Home Markings  
None

#### Liveness Properties

---

Dead Markings  
10 [31,30,27,26,25,...]

Dead Transition Instances  
None

Live Transition Instances  
None

#### Fairness Properties

---

No infinite occurrence sequences.